

Implementação do circuito quântico correspondente a uma matriz unitária 8×8 por Decomposição Cosseno-Seno (CSD)

PARTE 1. Descrição matemática do problema

Na decomposição Cosseno-Seno, a matriz unitária U é implementada a partir de um algoritmo recursivo. Assim, decomparamos U em um produto de matrizes, cada uma das quais é identificada com um novo tipo de porta que chamamos de rotação uniformemente controlada.

Com isso, seja $F_m^k(R_{\mathbf{a}})$ a representação de uma rotação uniformemente controlada. Ela é composta por k -rotações controladas do qubit m em torno do vetor tridimensional $\mathbf{a}$, ou seja, para cada um dos 2^k valores clássicos possíveis para os qubits de controle, teremos uma rotação. O sub-índice m pode adquirir qualquer um dos valores $1, 2, \dots, n$ e k os valores $1, 2, \dots, n-1$.

Por exemplo, a rotação controlada $F_{k+1}^k(R_{\mathbf{a}})$ tem representação

$$F_{k+1}^k(R_{\mathbf{a}}) = \begin{pmatrix} R_{\mathbf{a}}(\alpha_1) & & \\ & \ddots & \\ & & R_{\mathbf{a}}(\alpha_{2^k}) \end{pmatrix}, \quad (1)$$

em que $\alpha_1, \alpha_2, \dots, \alpha_{2^k}$ são ângulos que devem ser escolhidos livremente e a matriz de rotação $R_{\mathbf{a}}(\phi)$ é dada por

$$R_{\mathbf{a}}(\phi) = e^{i\mathbf{a} \cdot \boldsymbol{\sigma} \phi / 2} = I \cos \frac{\phi}{2} + i(\mathbf{a} \cdot \boldsymbol{\sigma}) \sin \frac{\phi}{2}. \quad (2)$$

Propomos uma implementação de $F_m^k(R_{\mathbf{a}})$ com $a_x = 0$ utilizando uma sequência alternada de 2^k portas CNOT e 2^k rotações de um qubit $R_a(\theta_i)$ atuando no qubit m . A posição do nó de controle na l -ésima porta CNOT é ajustada para corresponder à posição em que a $(l-1)$ -ésima e a l -ésima cadeias de bits, g_{l-1} e g_l , do código de Gray refletido binário diferem. Em códigos de Gray binários, as cadeias de bits adjacentes diferem, por definição, em apenas um único bit e, portanto, a posição é bem definida.

Na construção proposta, cada um dos qubits de controle regula um número par de portas NOT, visto que em um código de Gray cíclico cada bit é invertido um número par de vezes. Por outro lado, a equação acima resulta em

$$a_x = 0 \implies \sigma_x R_a(\theta) \sigma_x = R_a(-\theta) \quad (3)$$

Além disso, rotações posteriores em torno de qualquer eixo único $\mathbf{a}$ são aditivas, isto é, $R_a(\theta)R_a(\phi) = R_a(\theta + \phi)$ para ângulos arbitrários θ e ϕ . Assim, a construção resulta em uma rotação do qubit m em torno do eixo $\mathbf{a}$ por um ângulo que é uma combinação linear dos ângulos $\{\theta_i\}$. Consequentemente, o circuito quântico proposto é equivalente à porta $F_m^k(R_{\mathbf{a}})$, desde que os ângulos $\{\theta_i\}$ sejam uma solução do sistema linear de equações

$$M^k \begin{pmatrix} \theta_1 \\ \vdots \\ \theta_{2^k} \end{pmatrix} = \begin{pmatrix} \alpha_1 \\ \vdots \\ \alpha_{2^k} \end{pmatrix} \quad (4)$$

onde os elementos da matriz M_{ij}^k podem ser determinados usando a identidade $\sigma_x R_a(\theta) \sigma_x = R_a(-\theta)$.

Decomposição Cosseno-Seno (CSD)

A Decomposição Cosseno-Seno (CSD) é uma técnica para fatorar uma matriz unitária grande de tamanho $2^n \times 2^n$ em três blocos menores, aplicando essa divisão repetidamente até chegar a blocos simples de tamanho 2×2 .

1. A Divisão Principal A matriz original U é reescrita como o produto de três matrizes de blocos [1]:

$$U = \underbrace{\begin{pmatrix} u_{11}^1 & 0 \\ 0 & u_{12}^1 \end{pmatrix}}_{U_1^1} \underbrace{\begin{pmatrix} c_{11}^1 & s_{11}^1 \\ -s_{11}^1 & c_{11}^1 \end{pmatrix}}_{A_1^1} \underbrace{\begin{pmatrix} u_{21}^1 & 0 \\ 0 & u_{22}^1 \end{pmatrix}}_{\tilde{U}_2^1} \quad (5)$$

- **Estrutura:** As matrizes das pontas (U_1^1 e $\tilde{U}_2^1$) contêm blocos unitários em suas diagonais. A matriz do meio (A_1^1) armazena as rotações do sistema, formadas por senos e cossenos.
- **Entendendo os índices:** O índice superior indica o nível da etapa (nível de recursão); o índice inferior indica a posição da matriz na multiplicação final.
- **Flexibilidade:** A CSD não possui uma solução única. Podemos usar as simetrias internas da matriz U para escolher a forma mais simples possível.

2. O Conteúdo dos Blocos Em qualquer etapa i da divisão:

- Cada bloco u_{jk}^i é uma matriz unitária menor, de tamanho $2^{n-i} \times 2^{n-i}$.
- Os blocos c_{jk}^i e s_{jk}^i são matrizes diagonais com valores de cosseno $c_{jk}^i = \text{diag}_l(\cos \theta_l)$ e seno $s_{jk}^i = \text{diag}_l(\sin \theta_l)$.

Esses blocos organizam-se ao longo da diagonal principal da seguinte forma:

$$U_j^i = \text{diag}_k(u_{jk}^i), \quad (k = 1, \dots, 2^i) \quad (6)$$

$$A_j^i = \text{diag}_k \begin{pmatrix} c_{jk}^i & s_{jk}^i \\ -s_{jk}^i & c_{jk}^i \end{pmatrix}, \quad (k = 1, \dots, 2^{i-1}) \quad (7)$$

3. Repetição do Processo (Recursão) Podemos aplicar essa mesma regra de quebra aos blocos que restaram no passo anterior:

$$U_j^{i-1} = U_{2j-1}^i A_{\zeta(i,j)}^i \tilde{U}_{2j}^i \quad (8)$$

- **Organização das posições:** Para alinhar a contagem das matrizes ao longo dos níveis, utiliza-se a função de apoio $\zeta(i, j) = 2^{n-i-1}(2j - 1)$.
- **Ajustes extras:** Essa decomposição introduz 2^{n-1} graus de liberdade a mais do que a matriz original. Para fixar esses valores adicionais, definem-se matrizes diagonais unitárias P_j^i :

$$P_j^i = \text{diag}_k \left(p_{j, \lceil k/2 \rceil}^i \right)$$

Ao introduzir graus de liberdade adicionais na decomposição, conseguimos simplificar o circuito quântico e reduzir significativamente a quantidade total de portas necessárias.

4. Redução de Portas com Matrizes de Fase Definimos matrizes diagonais unitárias de fase $P_{jk}^i = \text{diag}_l(e^{i\phi_l})$. Como os ângulos ϕ_l podem ser escolhidos livremente, inserimos a identidade $I = P_{\zeta(i,j)}^i (P_{\zeta(i,j)}^i)^\dagger$ na equação recursiva:

$$U_j^{i-1} = U_{2j-1}^i P_{\zeta(i,j)}^i A_{\zeta(i,j)}^i U_{2j}^i \quad (9)$$

A parte $(P_{\zeta(i,j)}^i)^\dagger$ é absorvida pelo bloco U_{2j}^i , enquanto $P_{\zeta(i,j)}^i$ é mantida para simplificar as etapas seguintes. Esse processo exige que a recursão seja aplicada em uma ordem bem definida (do maior nível para o menor).

5. Estrutura Final do Circuito Quântico A multiplicação completa da matriz U assume a forma encadeada:

$$U = \left(\prod_{j=1}^{2^{n-1}-1} B_j A_j^{\gamma(j)} \right) B_{2^{n-1}} \quad (10)$$

Onde a função $\gamma(j)$ indica a posição do bit não nulo menos significativo na representação binária de j . Cada matriz dessa estrutura equivale a portas quânticas específicas:

- **Matrizes A_j^i :** São rotações uniformemente controladas no eixo y , denotadas por $F_i^{n-1}(R_y)$.
- **Blocos B_j :** Combinam rotações nos eixos z e y :

$$B_j = F_n^{n-1}(R_z) F_n^{n-1}(R_y) F_{\gamma(j)}^{n-1}(R_z) \quad (11)$$

- **Seção $(BA)_j$:** Unindo B_j e $A_j^{\gamma(j)}$, forma-se o bloco padrão repetido no circuito:

$$(BA)_j = F_n^{n-1}(R_z) F_n^{n-1}(R_y) F_{\gamma(j)}^{n-1}(R_z) F_{\gamma(j)}^{n-1}(R_y) \quad (12)$$

- **Bloco Final $B_{2^{n-1}}$:** Sem graus de liberdade restantes, este último bloco ajusta as fases do sistema e a fase global Φ :

$$B_{2^{n-1}} = F_n^{n-1}(R_z) F_n^{n-1}(R_y) [F_n^{n-1}(R_z) F_{n-1}^{n-2}(R_z) \dots F_1^0(R_z) \Phi] \quad (13)$$

6. Otimização e Complexidade Total Cada seção $(BA)_j$ contém duas rotações controladas em z e duas em y .

- **Truque do espelhamento:** Invertendo a ordem dos circuitos de rotação em y , eliminam-se 4 portas CNOT de cada uma das $2^{n-1} - 1$ seções.
- **Custo por seção:** Requer 2^{n+1} rotações simples de 1 qubit e $2^{n+1} - 4$ portas CNOT.
- **Custo do bloco final:** Espelhar o bloco $B_{2^{n-1}}$ cancela mais 2 portas CNOT.
- **Complexidade final do sistema:** Para sintetizar uma porta arbitrária de n qubits, o circuito precisa de $4^n - 2^{n+1}$ portas CNOT e 4^n portas de rotação de 1 qubit.

PARTE 2. Algoritmo para montagem do circuito

```
1 import numpy as np
2 import scipy.linalg as la
3 from scipy.linalg import cossin
4 from qiskit import QuantumCircuit
5 from qiskit.quantum_info import random_unitary
6 from qiskit.synthesis import OneQubitEulerDecomposer
7
8 # =====
9 # 1. FUNCOES AUXILIARES DE CODIGO DE GRAY E ROTACOES MULTIPLEXADAS
10 # =====
11 def gerar_codigo_gray(n_bits):
12     if n_bits == 0:
13         return ''
14     return [bin(i ^ (i >> 1))[2:].zfill(n_bits) for i in range(2**n_bits
15         )]
16
17 def identificar_bit_alterado(gray1, gray2):
18     """Retorna o indice do bit que mudou (da direita para a esquerda)."""
19     for idx, (b1, b2) in enumerate(zip(reversed(gray1), reversed(gray2))
20         ):
21         if b1 != b2:
22             return idx
23     return 0
24
25 def aplicar_multiplexed_r(qc, tipo_porta, angles, target_qubit,
26     control_qubits):
27
28     # Decompose uma rotacao multiplexada (Ry ou Rz) em uma sequencia
29     # alternada de CNOTs e rotacoes de 1 qubit usandoCodigo de Gray.
30     k = len(control_qubits)
31     if k == 0:
32         if tipo_porta == 'y':
33             qc.ry(angles[0], target_qubit)
34         else:
35             qc.rz(angles[0], target_qubit)
36         return
37
38     N = 2**k
39     gray_seq = gerar_codigo_gray(k)
40
41     # Construção da Matriz de Transformação do Código de Gray (Walsh-
42     # Hadamard)
43     M = np.zeros((N, N))
44     for i in range(N):
45         bits_i = [int(b) for b in bin(i)[2:].zfill(k)]
46         for j in range(N):
47             bits_g = [int(b) for b in gray_seq[j]]
48             prod = sum(bi * bg for bi, bg in zip(bits_i, bits_g))
49             M[i, j] = (-1) ** prod
50
51     # Calculo dos angulos alfas das portas individuais
52     alphas = (M.T @ angles) / N
53
54     # Aplicação da sequencia de portas CNOT e Rotacoes no circuito
55     for j in range(N):
```

```

52     if tipo_porta == 'y':
53         qc.ry(alphas[j], target_qubit)
54     else:
55         qc.rz(alphas[j], target_qubit)
56
57     # Determina qual qubit de controle altera o bit no Código de
58         Gray
59     proximo_j = (j + 1) % N
60     bit_mudar = identificar_bit_alterado(gray_seq[j], gray_seq[
        proximo_j])
    qc.cx(control_qubits[bit_mudar], target_qubit)

```

Listing 1: Funções auxiliares de Código de Gray e rotações multiplexadas

Este código implementa a decomposição de uma **rotação multiplexada** de 1 qubit (R_y ou R_z) controlada por k qubits em uma sequência alternada de rotações individuais e portas CNOT utilizando o **Código de Gray**.

1. Geração e Comparação do Código de Gray

- **gerar_codigo_gray(n_bits)**: Gera uma sequência binária de tamanho 2^k . A operação bit a bit $i \wedge (i \gg 1)$ garante a propriedade fundamental do código de Gray: duas cadeias de bits adjacentes diferem em exatamente um único bit.
- **identificar_bit_alterado(gray1, gray2)**: Compara duas cadeias consecutivas de Gray da direita para a esquerda (do bit menos significativo ao mais significativo). O índice do bit que mudou indica qual qubit da lista `control_qubits` deve atuar como controle na porta CNOT correspondente.

2. Cálculo dos Ângulos Individuais (α) Uma rotação multiplexada aplica um ângulo θ_i diferente no qubit alvo dependendo do estado dos k qubits de controle. O código converte o vetor de ângulos desejados θ nos ângulos α que serão aplicados pelas portas de 1 qubit do circuito:

- **Matriz M** : Constrói uma matriz $N \times N$ (onde $N = 2^k$) com elementos $M_{ij} = (-1)^{\mathbf{b}_i \cdot \mathbf{g}_j}$, relacionando o i -ésimo número binário $\mathbf{b}_i$ com o j -ésimo código de Gray $\mathbf{g}_j$.
- **Resolução do Sistema**: Como a matriz M é ortogonal até um fator de escala, o sistema $M\alpha = \theta$ é resolvido diretamente por:

$$\alpha = \frac{1}{2^k} M^T \theta$$

3. Montagem do Circuito (aplicar_multiplexed_r)

- **Caso Base ($k = 0$)**: Sem qubits de controle, aplica apenas uma rotação simples (`ry` ou `rz`) com o ângulo `angles[0]`.
- **Caso Geral ($k > 0$)**: Executa um loop de $j = 0$ até $2^k - 1$:
 1. Aplica a rotação de 1 qubit `ry(alphas[j])` ou `rz(alphas[j])` no `target_qubit`.
 2. Determina qual bit se altera ao passar da posição de Gray j para a posição $(j + 1) \bmod N$.
 3. Aplica uma porta `cx` (CNOT) tendo `control_qubits[bit_mudar]` como controle e `target_qubit` como alvo.

Etapa	Entrada / Operação	Saída / Efeito no Circuito
Ângulos Alvo	Vetor θ com 2^k ângulos	Define a operação multiplexada desejada
Transformação	$\alpha = \frac{1}{2^k} M^T \theta$	Ângulos das rotações de 1 qubit
Síntese	Iteração com $R_a(\alpha_j) + \text{cx}$	Circuito quântico equivalente com 2^k CNOTs

```

1 # =====
2 # 2. DESMULTIPLEXACAO DE BLOCOS DIAGONAIS (u1 XOR u2)
3 # =====
4 def demultiplex_block_diagonal(qc, u1, u2, qubits):
5     """
6     Fatoriza (u1 XOR u2) em matrizes nao controladas e uma rotacao
7     multiplexada Rz.
8     """
9     target = qubits[-1]
10    controls = qubits[:-1]
11
12    # Eigen-decomposicao de M = u1 @ u2^dagger
13    M = u1 @ u2.conj().T
14    vals, V = la.eig(M)
15    phases = np.angle(vals)
16    thetas = phases / 2.0
17
18    D = np.diag(np.exp(1j * thetas))
19    W1 = V
20    W2 = V.conj().T @ u2
21
22    # 1. Aplica W2 nos qubits de controle (Recursivo)
23    decompor_unitaria(qc, W2, controls)
24
25    # 2. Aplica Multiplexed Rz no qubit alvo
26    angles_rz = -2.0 * thetas
27    aplicar_multiplexed_r(qc, 'z', angles_rz, target, controls)
28
29    # 3. Aplica D e W1 nos qubits de controle (Recursivo)
30    decompor_unitaria(qc, D, controls)
31    decompor_unitaria(qc, W1, controls)

```

Listing 2: Desmultiplexação de blocos diagonais ($u_1 \oplus u_2$)

Esta função fatora uma matriz unitária em blocos diagonais da forma $U = u_1 \oplus u_2 = \begin{pmatrix} u_1 & 0 \\ 0 & u_2 \end{pmatrix}$ em operações sobre os qubits de controle (**controls**) e uma rotação multiplexada no qubit alvo (**target**).

1. Fundamento Matemático da Desmultiplexação Para separar a ação conjunta das matrizes u_1 e u_2 , o código busca uma transformação que relacione ambas as matrizes por meio de uma matriz diagonal de fases D :

- **Matriz de Transição (M):** Calcula-se $M = u_1 u_2^\dagger$. A auto-decomposição de M produz $M = V \Lambda V^\dagger$, onde $\Lambda = \text{diag}(e^{i2\theta_k})$.

- **Matrizes Intermediárias:**

- $D = \text{diag}(e^{i\theta_k})$ (matriz diagonal contendo as metades das fases de M).

- $W_1 = V$ (autovetores de M).
- $W_2 = V^\dagger u_2$.

- **Relação de Reconstrução:** Essa escolha garante que $u_2 = W_1 W_2$ e $u_1 = W_1 D^2 W_2$.

2. Passos de Execução do Código

1. **Separação de Qubits:** O último qubit da lista é definido como **target**, enquanto todos os anteriores atuam como **controls**.
2. **Decomposição Espectral (la.eig):** Obtém os autovalores λ_k e autovetores V da matriz M . Os ângulos θ_k são extraídos como metade das fases de λ_k .
3. **Construção de W_1 , W_2 e D :** Define os operadores unitários necessários para a reconstrução em termos de circuitos quânticos menores.
4. **Construção no Circuito (qc):**
 - Aplica W_2 recursivamente nos qubits de controle.
 - Aplica uma rotação $R_z(-2\theta)$ multiplexada no qubit alvo, condicionada pelos qubits de controle.
 - Aplica a correção de fase D e a transformação W_1 recursivamente nos qubits de controle.

Etapa	Operação Aplicada	Alvo	Finalidade
1	decompor_unitaria(W2)	controls	Prepara a base comum de entrada
2	aplicar_multiplexed_r('z', target (cond. por controls))	target	Alterna a fase relativa entre os blocos u_1 e u_2
3	decompor_unitaria(D)	controls	Ajusta o alinhamento de fase entre os blocos
4	decompor_unitaria(W1)	controls	Aplica a mudança de base final de saída

```

1 # =====
2 # 3. COMPILADOR RECURSIVO CSD (DECOMPOSICAO DE SHANNON)
3 # =====
4 def decompor_unitaria(qc, U, qubits):
5     """
6     Decompe recursivamente uma matriz unitaria U em portas fundamentais
7     """
8     dim = U.shape[0]
9
10    # CASO BASE: 1 Qubit (Matriz 2x2) -> Decomposicao Euler ZYZ (3
        rotacoes)
11    # CASO BASE: 1 Qubit (Matriz 2x2) decomposto do zero em Rz e Ry
12    if dim == 2:
13        # Normalizacao para SU(2) removendo fase global
14        fase = np.sqrt(1a.det(U))
15        U_norm = U / fase if fase != 0 else U
16
17        # Calculo dos angulos de Euler ZYZ
18        theta = 2 * np.arccos(np.clip(np.abs(U_norm[0, 0]), 0, 1))
19

```

```

20     angle1 = np.angle(U_norm[1, 1])
21     angle2 = np.angle(U_norm[1, 0])
22
23     phi = angle1 + angle2
24     lam = angle1 - angle2
25
26     qc.rz(lam, qubits[0])
27     qc.ry(theta, qubits[0])
28     qc.rz(phi, qubits[0])
29     return
30
31     # DECOMPOSICAO CSD PARA MATRIZES MAIORES (Ex: 8x8 para 3 qubits)
32     meio = dim // 2
33     (u1, u2), thetas_csd, (v1h, v2h) = cossin(U, p=meio, q=meio,
34         separate=True)
35
36     target = qubits[-1]
37     controls = qubits[:-1]
38
39     # Passo 1: Decompor bloco V_CSD (Direita)
40     demultiplex_block_diagonal(qc, v1h, v2h, qubits)
41
42     # Passo 2: Rotacao Multiplexada Ry central
43     angles_ry = 2.0 * thetas_csd
44     aplicar_multiplexed_r(qc, 'y', angles_ry, target, controls)
45
46     # Passo 3: Decompor bloco U_CSD (Esquerda)
47     demultiplex_block_diagonal(qc, u1, u2, qubits)

```

Listing 3: Compilador recursivo CSD (Decomposição de Shannon)

Esta função realiza a Decomposição Cosseno-Seno (CSD) de forma recursiva, compilando uma matriz unitária arbitrária U de dimensão $2^n \times 2^n$ em portas fundamentais de 1 qubit e CNOTs. O algoritmo resolve o problema reduzindo iterativamente o número de qubits até atingir o caso base de 1 qubit.

1. Caso Base: Decomposição Euler de 1 Qubit (dim = 2) Quando o algoritmo atinge uma submatriz de tamanho 2×2 , a decomposição de Euler no eixo ZYZ sintetiza o operador diretamente:

- **Normalização:** Remove a fase global dividindo a matriz por $\sqrt{\det(U)}$, convertendo $U \in U(2)$ para $SU(2)$ (ver mais detalhes no Apêndice 1).
- **Cálculo dos Ângulos ZYZ:** Extrai os ângulos de rotação θ , ϕ e λ a partir dos elementos de matriz:

$$\theta = 2 \arccos(|U_{0,0}|), \quad \phi = \angle(U_{1,1}) + \angle(U_{1,0}), \quad \lambda = \angle(U_{1,1}) - \angle(U_{1,0})$$

- **Síntese de Portas:** Insere no circuito a sequência de rotações $R_z(\lambda) \rightarrow R_y(\theta) \rightarrow R_z(\phi)$ no qubit correspondente.

2. Passo Recursivo: Decomposição CSD (dim > 2) Para $n > 1$ qubits, a função `scipy.linalg.cossin` fatora o operador U no produto $U = (u_1 \oplus u_2) \cdot A(\theta) \cdot (v_{1h} \oplus v_{2h})$. A construção do circuito ocorre da direita para a esquerda na multiplicação de matrizes:

- **Passo 1 (Bloco Direito $V^\dagger$):** Envia os blocos unitários v_{1h} e v_{2h} para a função `demultiplex_block_diagonal` que desmembra o bloco diagonal em sub-problemas menores.
- **Passo 2 (Rotação Central A):** O operador central de rotações de Givens é implementado como uma rotação multiplexada R_y no qubit alvo (`qubits[-1]`), controlada por todos os outros qubits (`qubits[:-1]`), com ângulos iguais a $2 \cdot \theta_{\text{csd}}$.
- **Passo 3 (Bloco Esquerdo U_L):** Envia os blocos unitários u_1 e u_2 para `demultiplex_block_diagonal`, finalizando a fatoração do nível atual da árvore recursiva.

Etapa no Código	Operação Algébrica	Estrutura Quântica Gerada
<code>cossin(U)</code>	$U \rightarrow (u_1 \oplus u_2) \cdot A(\theta) \cdot (v_{1h} \oplus v_{2h})$	Fatoração em 3 matrizes de tamanho $2^n \times 2^n$
<code>demultiplex(v1h, v2h)</code>	Decomposição de $(v_{1h} \oplus v_{2h})$	Chamadas recursivas nos qubits de controle
<code>aplicar_multiplexada</code>	Aplicação da matriz diagonal central A	Rotações R_y com CNOTs alternadas no target
<code>demultiplex(u1, u2)</code>	Decomposição de $(u_1 \oplus u_2)$	Chamadas recursivas nos qubits de controle

```

1 # =====
2 # 4. EXECUCAO, EXIBICAO DO CIRCUITO E CONTAGEM DE PORTAS
3 # =====
4
5 # Criar matriz unitaria aleatoria de 8x8 (3 qubits)
6 matriz_8x8 = random_unitary(8).data
7
8 # Inicializar circuito de 3 qubits
9 circuito_customizado = QuantumCircuit(3)
10
11 # Compilar a matriz 8x8 gerando o circuito do zero
12 decompor_unitaria(circuito_customizado, matriz_8x8, qubits=[0, 1, 2])
13
14 # A) Desenhar o circuito gerado
15 print("=== DESENHO DO CIRCUITO DECOMPOSTO DO ZERO ===")
16 print(circuito_customizado.draw(output='text'))
17
18 # B) Contar e analisar as portas do circuito
19 contagem = circuito_customizado.count_ops()
20 total_portas = sum(contagem.values())
21
22 print("\n=== ESTATISTICAS E CONTAGEM DE PORTAS ===")
23 for porta, qtd in contagem.items():
24     print(f"Porta '{porta}': {qtd}")
25
26 print(f"Total de portas no circuito: {total_portas}")
27 print(f"Quantidade total de CNOTs (cx): {contagem.get('cx', 0)}")
28 print(f"Profundidade do circuito: {circuito_customizado.depth()}")

```

Listing 4: Execução, exibição do circuito e contagem de portas

Este bloco de código é a etapa final do pipeline: ele gera uma matriz unitária aleatória de 3 qubits, executa o compilador recursivo CSD construído anteriormente, desenha o circuito gerado e analisa suas métricas de desempenho.

1. Geração da Matriz e Inicialização do Circuito

- `random_unitary(8).data`: Cria uma matriz unitária 8×8 (representando um operador de 3 qubits, já que $2^3 = 8$) com distribuição Haar uniforme.
- `QuantumCircuit(3)`: Instancia um circuito quântico vazio no Qiskit com 3 qubits (índices 0, 1 e 2).

2. Compilação Quântica (`decompor_unitaria`)

- Chama a função principal de decomposição passando a matriz 8×8 , o circuito e a lista de qubits.
- **Execução Interna**: A matriz é fatorada via CSD em blocos diagonais, rotações multiplexadas R_y e R_z , e portas de 1 qubit, convertendo a matriz densa em uma sequência de portas `cx`, `ry` e `rz`.

3. Visualização e Profiling do Circuito

- `draw(output='text')`: Renderiza uma diagramação em arte ASCII no terminal exibindo a linha de tempo de cada qubit e as portas aplicadas.
- `count_ops()`: Retorna um dicionário com a frequência de cada porta quântica presente no circuito (ex: `{ 'cx': N, 'rz': M, 'ry': K }`).
- `depth()`: Calcula a profundidade (depth) do circuito, que representa o número de passos de tempo paralelos necessários para executar o circuito em um hardware real.

Resumo das Métricas Extraídas

Métrica no Código	Função / Origem	O que mede
Desenho ASCII	<code>circuito_customizado.draw()</code>	Estrutura visual e conectividade das portas
Contagem de CNOTs	<code>contagem.get('cx', 0)</code>	Custo de 2 qubits (principal fonte de ruído em hardware real)
Total de Portas	<code>sum(contagem.values())</code>	Volume total de operações ($1Q + 2Q$)
Profundidade	<code>circuito_customizado.depth()</code>	Comprimento do caminho crítico de execução

Conclusão

Ao comparar a estrutura do circuito impresso com a especificação teórica da CSD de 48 CNOTs da Tabela 1 em [2], percebe-se que a lógica de sequenciamento pelo Código de Gray e o controle das rotações multiplexadas atingiram o rendimento máximo da variante CSD clássica. A diferença operacional entre o resultado prático e a CSD ideal reside na sobretaxa de portas de 1 qubit e na profundidade. O circuito construído contabiliza 204 portas no total (com 104 R_z e 52 R_y) e uma profundidade de 177, por manter rotações com ângulos residuais ou nulos (como $R_y(0)$ e $R_y(2,9802 \times 10^{-8})$) e não fundir portas R_z consecutivas no mesmo qubit, enquanto o modelo analítico da CSD assume a eliminação prévia dessas operações redundantes.

A $SU(2)$

$SU(2)$ (Grupo Especial Unitário de grau 2) é o grupo de Lie formado por todas as matrizes complexas 2×2 unitárias que possuem determinante igual a 1. Na computação e mecânica quântica, ele representa o conjunto de todas as operações e rotações possíveis em um único qubit (desconsiderando a fase global).

Definição Matemática

$$SU(2) = \{U \in \mathbb{C}^{2 \times 2} : U^\dagger U = I, \det(U) = 1\}$$

Propriedades Fundamentais

- **Geradores:** É gerado pelas matrizes de Pauli $(\sigma_x, \sigma_y, \sigma_z)$. Qualquer elemento de $SU(2)$ pode ser expresso como uma rotação em torno de um eixo unitário $\mathbf{n}$:

$$U(\theta, \mathbf{n}) = \exp\left(-i\frac{\theta}{2}\mathbf{n} \cdot \boldsymbol{\sigma}\right) = \cos\left(\frac{\theta}{2}\right)I - i\sin\left(\frac{\theta}{2}\right)(\mathbf{n}_x\sigma_x + \mathbf{n}_y\sigma_y + \mathbf{n}_z\sigma_z)$$

- **Relação com a Esfera de Bloch:** Como $SU(2)$ descreve rotações no espaço tridimensional, qualquer porta quântica de 1 qubit corresponde geometricamente a girar o vetor de estado sobre a superfície da Esfera de Bloch.
- **Duplo Recobrimento de $SO(3)$:** É o *double cover* do grupo de rotações 3D reais ($SO(3)$). Isso significa que girar um sistema de spin-1/2 em 360° (2π) resulta na matriz $-I$, exigindo uma rotação de 720° (4π) para retornar ao estado original I .
- **Geometria e Quatérnios:** É topologicamente equivalente à esfera tridimensional S^3 no espaço 4D e é isomorfo ao grupo dos quatérnios de norma unitária.

Papel na Computação Quântica Uma matriz unitária geral de 1 qubit em $U(2)$ possui 4 parâmetros livres (3 rotações mais 1 fase global $e^{i\phi}$). Ao dividir pela fase global — que é fisicamente inobservável —, a operação se reduz a um elemento de $SU(2)$ com apenas 3 parâmetros reais, exatamente os ângulos de Euler (θ, ϕ, λ) utilizados no caso base do compilador.

Referências

- [1] Möttönen, M., Vartiainen, J. J., Bergholm, V., & Salomaa, M. M. (2004). Quantum circuits for general multiqubit gates. *Physical Review Letters*, 93(13), 130502. <https://doi.org/10.1103/PhysRevLett.93.130502>
- [2] Krol, Anneriet & Al-Ars, Zaid. (2024). Beyond quantum Shannon decomposition: Circuit construction for n -qubit gates based on block-ZXZ decomposition. *Physical Review Applied*, 22, 034019. <https://doi.org/10.1103/PhysRevApplied.22.034019>